Deep Learning-Assisted Pre-Filtering for Selective Graph-Structured LLM-Based Vulnerability Detection

Gia Khanh Tran, Minh Dung Vo, and Dai Tho Nguyen *
VNU University of Engineering and Technology, Hanoi, Vietnam
* Contact author: nguyendaitho@vnu.edu.vn

***Abstract*—Large language models (LLMs) can improve function-level vulnerability detection when combined with structural program evidence and retrieved demonstrations, but invoking an LLM for every function is expensive. We introduce a calibrated multi-view selective GRACE-style pipeline that routes each function to a low-risk skip branch, a direct high-risk branch, or graph-conditioned LLM inspection.**

In a descriptive, non-paired local comparison with our reproduced single-run GRACE-style baseline, mean positive-class F1 and recall, averaged across the three datasets, are higher by 0.0986 and 0.1041 (recall is higher on Devign and Big-Vul but lower on ReVeal), while the fraction of test functions resolved before downstream LLM inference averages 34.59% (equal-weight across the three datasets).

Across 25 run–seed measurements per dataset, Devign, Big-Vul, and ReVeal yield F1 scores of 0.6667 ± 0.0053, 0.4011 ± 0.0028, and 0.5221 ± 0.0052, with corresponding LLM-call ratios of $16.37\% \pm 3.26\%$, $93.18\% \pm 0.90\%$, and $86.68\% \pm 0.74\%$. A retained five-seed Devign analysis exposes a recall-oriented operating point (specificity 0.3014 ± 0.0453). These results support selective LLM use as a security-triage mechanism with strongly dataset-dependent reductions in downstream LLM usage.

***Index Terms*—software vulnerability detection, large language models, graph-based code analysis, selective inference, calibration, in-context learning**

I. Introduction

Automated software vulnerability detection remains challenging because vulnerable code patterns are sparse, context-dependent, and often expressed through control-flow, data-flow, and API-usage relations rather than isolated lexical tokens [1]–[3]. Graph-based representations and fine-grained localization can expose security-relevant evidence that whole-function token sequences dilute [4]–[6].

Large language models offer a complementary direction that combines broad code knowledge with retrieved in-context examples [7], [8]; GRACE, for example, combines graph-structured code information with retrieved demonstrations for vulnerability detection [9]. However, a non-selective GRACE-style pipeline must extract structural evidence, retrieve demonstrations, build a prompt, and invoke the LLM for every function—an efficiency bottleneck for large-scale screening.

We investigate whether a front-end intended to reduce downstream LLM use can reserve graph-conditioned LLM reasoning for uncertain cases, combining a multi-view prefilter, post-hoc calibration, and three-way routing before a GRACE-style inspection stage. Selective classification, probability calibration, and cascade routing are established ideas [19]–[21], [23]; our contribution is their integration with complementary code views and a graph/retrieval/LLM vulnerability-analysis pipeline, together with explicit effectiveness–downstream-LLM-use and branch-error measurements.

The main contributions are: (1) a multi-view prefilter over lexical, AST-like, semantic, and graph-informed features; (2) validation-calibrated three-way routing that makes the vulnerability-coverage versus downstream-LLM-use trade-off explicit; and (3) evaluation on Devign, Big-Vul, and ReVeal with repeated multi-seed robustness analysis comprising five repetitions of a shared five-seed set per dataset, together with confidence intervals, routing thresholds, calibration diagnostics, and branch-level error analysis for one retained detailed five-seed Devign repetition. Published GRACE results serve only as literature context; comparative statements are based on a descriptive, non-paired local comparison with a reproduced GRACE-style baseline. Source code is available at https://github.com/khanhtran0111/VulGuardVN.

II. Related Work

Classical learning-based vulnerability detectors differ mainly in the code representation they use. VulDeePecker learns from semantically related code gadgets with deep sequence modeling [10]; Devign performs graph-level classification over composite program graphs using graph neural networks [1]; DeepWukong builds deep graph embeddings for static vulnerability detection [11]; IVDetect adds fine-grained vulnerable-statement interpretations based on dependency contexts [4]; LineVul moves to transformer-based line-level prediction [6]; and VulBERTa shows that lightweight source-code pre-training can be highly competitive for vulnerability detection [12]. These works motivate our representation choices but do not formulate inference as a selective routing problem.

LLM-based vulnerability detection has progressed from direct prompting to increasingly structured reasoning. Early studies found generic LLM prompting unreliable on vulnerability tasks [13], [14]; subsequent work improved performance with prompt engineering and auxiliary structural cues [15],

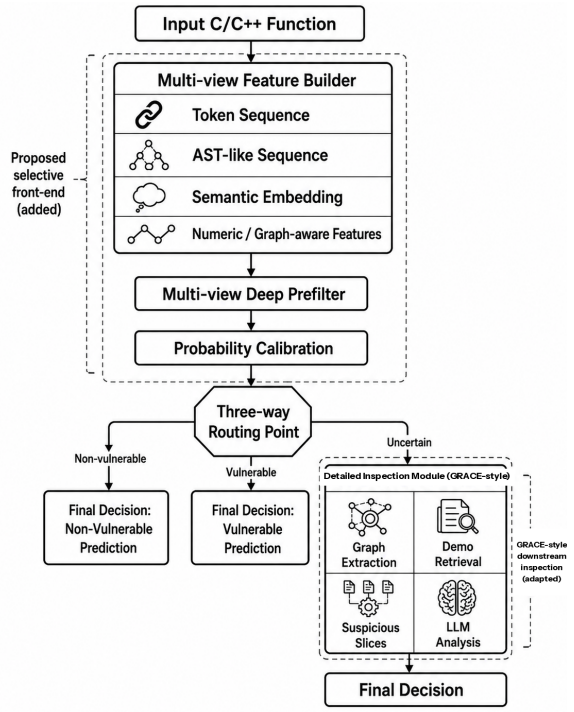

Fig. 1. Selective GRACE-style vulnerability-detection pipeline.

graph- and demonstration-guided reasoning in GRACE [9], hybrid deep-learning-plus-LLM prompting in DLAP [16], neuro-symbolic whole-repository reasoning in IRIS [17], and knowledge-level retrieval augmentation in Vul-RAG [18]. Orthogonally, selective prediction formalizes risk–coverage tradeoffs and reject options [19], [20], while calibration converts model scores into usable probabilities for such policies [21]–[23]. Our method combines these lines by using a calibrated selective front-end to decide when GRACE-style inspection is necessary.

III. Selective GRACE-style Framework

This section presents the proposed selective GRACE-style framework, which augments a GRACE-style downstream inspection stage with a calibrated multi-view prefilter and a three-way routing controller. Under this design, function-level vulnerability detection is cast as a selective inference problem that jointly considers predictive performance, calibrated confidence, and downstream inference cost [19].

A. Overview

Figure 1 summarizes the architecture: the proposed additions are the multi-view feature builder, learned prefilter, post-hoc calibration layer, and three-way routing controller; the preserved GRACE-style component is the downstream *inspect* branch, which combines graph-structured evidence, retrieved demonstrations, and LLM reasoning.

Let x denote an input function and $y \in \{0, 1\}$ its ground-truth label, where $y = 1$ indicates a vulnerable function. The selective front-end constructs four complementary views, $V(x) = \{v_{\mathrm{tok}}(x), v_{\mathrm{ast}}(x), v_{\mathrm{sem}}(x), v_{\mathrm{num}}(x)\}$, where $v_{\mathrm{tok}}(x)$ is the token sequence, $v_{\mathrm{ast}}(x)$ is an AST-like structural sequence, $v_{\mathrm{sem}}(x)$ is a semantic code embedding, and $v_{\mathrm{num}}(x)$ contains graph-informed numerical features. To keep the front-end lightweight, $v_{\mathrm{num}}(x)$ stores compact graph-aware summaries rather than the richer graph bundle reserved for the downstream inspection branch.

B. Multi-view Prefilter

The prefilter processes three learned views and one numeric view. The token, AST-like, and semantic branches produce latent representations

$$h_{\mathrm{tok}}(x) = E_{\mathrm{tok}}(v_{\mathrm{tok}}(x)), \quad h_{\mathrm{ast}}(x) = E_{\mathrm{ast}}(v_{\mathrm{ast}}(x)), \quad h_{\mathrm{sem}}(x) = E_{\mathrm{sem}}(v_{\mathrm{sem}}(x)), \tag{1}$$

where $E_{\mathrm{tok}}(\cdot)$ and $E_{\mathrm{ast}}(\cdot)$ are sequence encoders, and $E_{\mathrm{sem}}(\cdot)$ is a projection layer over a pretrained semantic embedding.

To preserve branch-specific risk signals for calibration and routing, the prefilter also computes two auxiliary scores, $p_{\mathrm{sem}}(x) = \sigma(g_{\mathrm{sem}}(h_{\mathrm{sem}}(x)))$ and $p_{\mathrm{gra}}(x) = \sigma(g_{\mathrm{gra}}(v_{\mathrm{num}}(x)))$, where $\sigma(\cdot)$ denotes the sigmoid function, $g_{\mathrm{sem}}(\cdot)$ is a semantic scoring head, and $g_{\mathrm{gra}}(\cdot)$ is a graph/numeric scoring head.

We then define the fused representation as

$$z(x) = [\, h_{\mathrm{tok}}(x);\, h_{\mathrm{ast}}(x);\, h_{\mathrm{sem}}(x);\, v_{\mathrm{num}}(x);\, p_{\mathrm{sem}}(x);\, p_{\mathrm{gra}}(x)\,]. \tag{2}$$

The fusion network then outputs a raw vulnerability score and the corresponding uncalibrated probability.

$$s_{\mathrm{fus}}(x) = f_\theta(z(x)), \qquad p_{\mathrm{fus}}(x) = \sigma(s_{\mathrm{fus}}(x)). \tag{3}$$

The prefilter parameters θ are learned on the training split using standard binary cross-entropy. The auxiliary scores $p_{\mathrm{sem}}(x)$ and $p_{\mathrm{gra}}(x)$ are branch-specific signals fused into $z(x)$; routing uses only the calibrated $\hat{p}(x)$ defined below, not the uncalibrated $p_{\mathrm{fus}}(x)$. For brevity, Algorithm 1 denotes the computation of $z(x)$ in Eq. (2) by FuseViews.

C. Calibration and Three-Way Routing

Because routing is threshold-based, it should operate on a calibrated probability rather than an uncalibrated model output. Modern neural predictors often rank well yet remain poorly calibrated, making fixed routing thresholds overly aggressive or overly conservative. We therefore apply post-hoc calibration on a shared validation/calibration split $\mathcal{D}_{\mathrm{cal}}$ disjoint from the gradient-training records [21]–[23]. The same split also drives early stopping (best-weight restoration) and learning-rate scheduling during training and is later reused for calibrator fitting/selection and threshold selection; this reuse can make the selected calibrator and thresholds slightly optimistic, and only the test split remains untouched.

Post-hoc calibration maps the uncalibrated fused probability to a routing probability, $\hat{p}(x) = C(p_{\mathrm{fus}}(x))$. In the default `auto` mode, the implementation fits four candidate calibrators on the calibration split—Platt scaling, temperature scaling, isotonic regression, and beta calibration—and selects the one with

the lowest negative log-likelihood on that same split, breaking ties by expected calibration error and then Brier score; this selection is in-sample and can favor flexible calibrators such as isotonic regression. In the retained detailed Devign repetition (Section V-B), this rule selected isotonic regression in all five seed evaluations.

Given routing thresholds $0 \le \tau_{\text{low}} < \tau_{\text{high}} \le 1$, we define

$$r(x) = \begin{cases} \textsc{Skip}, & \hat{p}(x) \le \tau_{\text{low}}, \\ \textsc{High-Risk}, & \hat{p}(x) \ge \tau_{\text{high}} \ \wedge \ \delta_{\text{high}} = 1, \\ \textsc{Inspect}, & \text{otherwise.} \end{cases} \tag{4}$$

The thresholds τ_{low} and τ_{high} are selected on the validation split and fixed before testing. In the verified Devign configuration, τ_{low} is chosen to satisfy a target recall of 0.995, while τ_{high} maximizes validation F1 subject to $\tau_{\text{high}} > \tau_{\text{low}}$ and the direct-accept minimum $\tau_{\text{high}} \ge 0.20$. Unless otherwise stated, we use $\delta_{\text{high}} = 1$, so $\hat{p}(x) \ge \tau_{\text{high}}$ is accepted directly as vulnerable. Seed-specific thresholds are reported in Section V-B. For brevity, Algorithm 1 denotes the calibration operation by CALIBRATE.

D. GRACE-Style Downstream Inspection and Attribution

Only *inspect*-branch functions reach the downstream stage, which follows the GRACE-style principle of combining graph-structured program evidence, retrieved in-context demonstrations, and LLM-based reasoning [9]. The term *GRACE-style* indicates that this stage preserves GRACE's main functional ideas rather than exactly reproducing its implementation details, graph backends, prompts, or language models.

The downstream stage proceeds in four steps. EXTRACTGRAPH obtains graph evidence (AST, control flow, data dependencies) from a code-property-graph-style representation when available, else a fallback structural extractor. LOCALIZESUSPICIOUSSLICE identifies compact, likely security-relevant regions so that the prompt is not dominated by the entire function body. RETRIEVEDEMONSTRATIONS selects relevant labeled examples from the demonstration pool for in-context reasoning. INSPECT then builds a graph-conditioned prompt from the function, graph evidence, suspicious slices, and retrieved demonstrations, and queries the local code LLM for the final vulnerability decision. Formally, for an inspected function x,

$$\begin{aligned} G_x &\leftarrow \textsc{ExtractGraph}(x), \\ S_x &\leftarrow \textsc{LocalizeSuspiciousSlice}(x, G_x), \\ D_x &\leftarrow \textsc{RetrieveDemonstrations}(x, S_x), \\ (\tilde{y}_x, A_x) &\leftarrow \textsc{Inspect}(x, G_x, S_x, D_x), \end{aligned} \tag{5}$$

where G_x denotes graph-structured evidence, S_x denotes suspicious slices, D_x denotes retrieved demonstrations, $\tilde{y}_x$ is the downstream prediction, and A_x denotes optional attribution information returned for human inspection. The attribution is not a standalone explanation guarantee; it records the graph evidence, slices, and prompt-level evidence used by the inspection stage.

Algorithm 1 Selective GRACE-style inference

Require: Function x, calibrator C, thresholds τ_{low} and τ_{high}, direct-high flag δ_{high}
Ensure: Prediction $\hat{y}$, decision source d, optional attribution A_x

```
Build v_tok(x), v_ast(x), v_sem(x), and v_num(x)
Form the fused representation z(x) ← FUSEVIEWS(x)
Compute the fused probability p_fus(x) ← σ(f_θ(z(x)))
Calibrate the probability p̂(x) ← C(p_fus(x))
if p̂(x) ≤ τ_low then
    return (non-vulnerable, PREFILTER, ∅)
else if p̂(x) ≥ τ_high and δ_high = 1 then
    return (vulnerable, PREFILTER, ∅)
else
    G_x ← EXTRACTGRAPH(x)
    S_x ← LOCALIZESUSPICIOUSSLICE(x, G_x)
    D_x ← RETRIEVEDEMONSTRATIONS(x, S_x)
    (ŷ, A_x) ← INSPECT(x, G_x, S_x, D_x)
    return (ŷ, LLM, A_x)
end if
```

Algorithm 1 summarizes the complete inference procedure, separating the selective front-end from the GRACE-style downstream stage: low-risk samples are resolved without LLM inference; high-risk samples can be accepted directly or optionally verified, depending on the selected policy; and only the remaining samples invoke the downstream inspection module.

IV. EVALUATION PROTOCOL

Unless stated otherwise, all reported numbers come from our local implementation of the selective GRACE-style pipeline, where *GRACE-style* is used in the sense of Section III-D.

A. Benchmarks and Dataset Statistics

We evaluate on Devign, Big-Vul, and ReVeal, three widely used function-level vulnerability-detection benchmarks [1], [3], [9], [24]. Per the dataset statistics reported in the original GRACE study, the Devign benchmark corresponds to the FFmpeg+Qemu dataset and contains 22,361 functions (10,067 vulnerable, 12,294 non-vulnerable). Big-Vul contains 179,299 functions (10,547 vulnerable, 168,752 non-vulnerable), and ReVeal contains 18,169 functions (1,664 vulnerable, 16,505 non-vulnerable).

B. Split Construction and Calibration Protocol

For Devign and Big-Vul, we construct reproducible grouped-stratified train/validation/test splits with a target ratio of 8:1:1, corresponding to approximately 80% training data, 10% validation data, and 10% test data. Concretely, ten-fold stratified group splitting yields the test fold, and an inner nine-fold grouped stratification of the remainder yields the validation fold, leaving the rest for training.

The grouping key is a canonicalized source-code hash, denoted `code_hash`, obtained from normalized function text and used only for split construction. This prevents duplicated or trivially reformatted functions from crossing split boundaries.

For ReVeal, we preserve the released train/validation/test split when available, or valid source-provided split assignments otherwise; failing both, ReVeal uses the same grouped-stratified procedure as Devign and Big-Vul.

Because grouped stratification must satisfy label-stratification and group-separation constraints simultaneously, realized split sizes may deviate slightly from the target ratio; the *Samples* column therefore reports the actual number of test records evaluated.

Calibration parameters and routing thresholds are selected on validation data without test labels and are fixed before test-set evaluation (Section III-C).

C. Experimental Configuration

The default semantic encoder is UniXcoder [25]. Graph extraction is requested in `auto` mode: when a Joern-based code-property-graph backend is available, it is used; otherwise, the system falls back to a heuristic structural extractor. This follows prior work showing that structural representations encode syntax, control flow, and data dependence beyond token-only views [1], [2].

Graph-backend status is retained when available. In the one detailed five-seed Devign repetition, Joern was unavailable and the heuristic fallback was used. The repeated report includes run–seed metrics for all three datasets but no complete per-record logs beyond that repetition; we therefore make no backend-specific post-hoc claims for those measurements.

The learned prefilter is configured for up to 10 epochs with a batch size of 128 and a learning rate of 7×10^{-4}. The downstream local LLM is Qwen2.5-Coder-7B-Instruct loaded in 4-bit quantized form [26]. The reported configuration uses `inspect_demos`=2, `high_risk_demos`=1, and `max_new_tokens`=96.

All experiments ran on Kaggle notebooks with two NVIDIA Tesla T4 GPUs (16 GB each). Under this configuration, a single end-to-end run takes approximately 3 hours on Devign and ReVeal and approximately 24 hours on Big-Vul. The longer Big-Vul runtime reflects its larger size and the high fraction of functions forwarded to LLM inspection (Section V-C).

D. Repeated Multi-Seed Robustness Protocol

To quantify run-to-run and seed-level robustness, we conduct five separately executed repetitions per dataset, each evaluating the shared seed set $\{1, 7, 21, 42, 100\}$, for 25 run–seed measurements per dataset. Every measurement reruns the applicable split construction, feature generation, prefilter training, calibration, threshold selection, and test evaluation. Between-seed variation includes prefilter initialization and the realized test partitions, which differed across seeds in the retained measurements; residual differences between repetitions sharing a seed reflect execution-level nondeterminism that was not separately controlled. LLM generation uses greedy decoding (`temperature`=0, `do_sample`=False), excluding stochastic sampling as a source. Aggregate per-run–seed metric summaries are available at `outputs/runseed_metrics.csv` in the public repository.

ReVeal preserves a released or source-provided split when available and otherwise uses grouped stratification (Section IV-B); the measurements thus capture complete-pipeline variation, not repeated initializations on one fixed test partition.

We report descriptive mean±SD over all 25 measurements per dataset (Table I). Because the design is hierarchical (five repetitions of five seeds), the rows are not treated as mutually independent samples and no confirmatory significance claim is made; only an exploratory seed-level consistency check is reported in Section V. As a descriptive sensitivity check, aggregating repetitions within each seed yields F1 SDs of 0.0041, 0.0020, and 0.0037 for Devign, Big-Vul, and ReVeal, respectively; no inferential conclusion is drawn from this comparison. Per-record artifacts exist only for one retained five-seed Devign repetition, enabling its calibration, confusion-matrix, threshold, and branch analyses; unavailable per-example quantities are not inferred for the other measurements.

E. Comparison Scope

Published GRACE values [9] are external literature context: their preprocessing, splits, prompts, graph backend, model, and runtime are not protocol-aligned with ours, so they support neither paired comparisons nor causal improvement claims. Local comparisons instead use our reproduced GRACE-style baseline: implementation and protocol are shared, while the evaluation itself is descriptive and non-paired (Section V-D).

F. Evaluation Metrics

We report accuracy, positive-class precision P, recall R, $F1 = 2PR/(P + R)$, ROC-AUC, and PR-AUC. Accuracy, precision, recall, and F1 use final pipeline decisions, whereas ROC-AUC and PR-AUC use calibrated prefilter probabilities (*PF* in Table I). Repeated multi-seed classification and routing metrics use descriptive mean±SD across 25 run–seed measurements per dataset. For the retained five-seed Devign subset, per-record artifacts additionally provide specificity, balanced accuracy, macro-F1, Matthews correlation coefficient (MCC), branch errors, Brier score, negative log-likelihood (NLL), and expected calibration error (ECE).

Routing uses the *skip*, *high*, and *inspect* fractions; the LLM-call ratio—the test fraction sent to downstream inspection—is the primary operational cost indicator. Record-level timing excludes precomputed feature-store construction, so the ratio is a downstream-inference proxy, not complete end-to-end cost.

V. Results and Discussion

A. Main Results

Table I presents the main comparison: a descriptive, non-paired local comparison against the reproduced GRACE-style baseline (published GRACE values are literature context only; Section IV-E), with the selective variant summarized as mean±SD over the 25 run–seed measurements of Section IV-D. Relative to the fixed single-run baseline values, mean F1 is higher on all three datasets, by $+0.0800$, $+0.0826$, and $+0.1331$ on Devign, Big-Vul, and ReVeal; the three-dataset average differences are $+0.0986$ in positive-class F1 and $+0.1041$ in recall, while the fraction of test functions resolved before downstream LLM inference averages 34.59% (an equal-weight average across the three datasets, not a sample-pooled fraction).

TABLE I
MAIN RESULTS ON DEVIGN, BIG-VUL, AND REVEAL.

Dataset	Method	Samples	Acc.	Prec.	Rec.	F1	PF ROC-AUC[f]	PF PR-AUC[f]	LLM ratio
Devign	Reproduced baseline[a]	2,726	0.5149	0.4925	0.7253	0.5867	–	–	1.000[b]
Devign	Selective (multi-seed)[c]	2,732	0.5827 ± 0.0106	0.5226 ± 0.0053	0.9210 ± 0.0235	0.6667 ± 0.0053[d]	0.6985 ± 0.0049	0.6346 ± 0.0156	0.1637 ± 0.0326
Devign	GRACE reference[e]	–	0.5978	0.5394	0.8213	0.6511	–	–	1.000[b]
Big-Vul	Reproduced baseline[a]	21,709	0.8896	0.2980	0.3420	0.3185	–	–	1.000[b]
Big-Vul	Selective (multi-seed)[c]	21,712	0.9186 ± 0.0006	0.3350 ± 0.0044	0.5000 ± 0.0079	0.4011 ± 0.0028[d]	0.8421 ± 0.0029	0.3890 ± 0.0055	0.9318 ± 0.0090
Big-Vul	GRACE reference[e]	–	0.9073	0.3252	0.3908	0.3550	–	–	1.000[b]
ReVeal	Reproduced baseline[a]	2,274	0.8440	0.3030	0.5430	0.3890	–	–	1.000[b]
ReVeal	Selective (multi-seed)[c]	2,274	0.8493 ± 0.0047	0.5448 ± 0.0123	0.5017 ± 0.0144	0.5221 ± 0.0052[d]	0.8293 ± 0.0044	0.4225 ± 0.0055	0.8668 ± 0.0074
ReVeal	GRACE reference[e]	–	0.8973	0.3321	0.6153	0.4313	–	–	1.000[b]
Dataset avg.	Reproduced baseline[a]	–	0.7495	0.3645	0.5368	0.4314	–	–	1.000
Dataset avg.	Selective (multi-seed)[c]	–	0.7835	0.4675	0.6409	0.5300[d]	0.7900	0.4820	0.6541
Dataset avg.	GRACE reference[e]	–	0.8008	0.3989	0.6091	0.4791	–	–	1.000

[a] Our fixed single-run local GRACE-style reference. The comparison is non-paired: Devign and Big-Vul share the split policy but not identical realized test partitions, whereas ReVeal uses the same released split.
[b] The LLM-call ratio of non-selective GRACE-style inference is set to 1.000 by convention.
[c] Mean±SD over 25 run–seed measurements (five repetitions × seeds $\{1, 7, 21, 42, 100\}$); *Samples* is the mean realized test size; *Dataset avg.* rows are equal-weight means of the three datasets, reported without SD because a cross-dataset SD would depend on pairing run–seed indices across independent datasets.
[d] ΔF1 versus the reproduced baseline: $+0.0800$ (Devign), $+0.0826$ (Big-Vul), $+0.1331$ (ReVeal); dataset-averaged ΔF1 $+0.0986$; ΔRecall $+0.1041$.
[e] Published GRACE values [9]; external literature context only, not protocol-aligned with our runs.
[f] Computed from calibrated prefilter probabilities (the continuous score defined uniformly for all test records), reflecting prefilter discrimination rather than final branch decisions; baseline values are omitted as its probability source is not directly comparable.

The recall difference is not uniform: $+0.1957$ (Devign) and $+0.1580$ (Big-Vul) but -0.0413 on ReVeal, where precision rises from 0.3030 to 0.5448 ± 0.0123—the selective variant shifts ReVeal's precision–recall operating point rather than uniformly dominating. As exploratory evidence of consistency, the seed-level F1 differences from the single-run baseline value (averaging repetitions within each seed, n=5) are positive for every seed on all three datasets; since this fixed comparator has unmeasured variability and only five differences exist, we report direction consistency only, not a two-method hypothesis test.

Devign has the lowest downstream LLM-use ratio ($16.37\% \pm 3.26\%$), whereas Big-Vul and ReVeal still send $93.18\% \pm 0.90\%$ and $86.68\% \pm 0.74\%$ of test functions to the LLM. Thus the reduction in downstream LLM usage is not uniform, and high accuracy on the strongly imbalanced Big-Vul set should not be interpreted as sufficient evidence of detection quality by itself.

One unverified hypothesis consistent with the observed routing ratios is the following. Because τ_{low} is selected under a high recall target, it must lie below the calibrated probabilities of nearly all vulnerable functions. On the close-to-balanced Devign set, the selected thresholds place most records in the direct high-risk branch (Table II); on the strongly imbalanced Big-Vul and ReVeal sets, calibrated probabilities may concentrate between the two thresholds—the recall constraint leaves little mass below τ_{low} and few functions exceed τ_{high}—so most functions remain in the inspect region. Direct verification requires per-example score histograms from per-record artifacts retained only for one Devign repetition (Section V-D).

B. Multi-Seed Robustness and Devign Diagnostics

In the retained five-seed Devign repetition with preserved per-record artifacts, Accuracy is 0.5827 ± 0.0103, Precision 0.5226 ± 0.0054, Recall 0.9214 ± 0.0255, F1 0.6668 ± 0.0051, ROC-AUC 0.6985 ± 0.0055, and PR-AUC 0.6353 ± 0.0154.

TABLE II
DEVIGN BRANCH-LEVEL ERROR AUDIT.

Branch	Coverage	Primary error	Error rate
Skip	$4.36 \pm 1.39\%$	false negative	$3.51 \pm 1.88\%$
High	$79.26 \pm 3.87\%$	false positive	$47.55 \pm 0.61\%$
Inspect	$16.37 \pm 3.46\%$	final misclass.	$23.29 \pm 2.64\%$

The corresponding 95% t-intervals from the five seed-level measurements are $[0.5699, 0.5955]$ (Accuracy), $[0.6605, 0.6731]$ (F1), $[0.6917, 0.7053]$ (ROC-AUC), and $[0.6162, 0.6544]$ (PR-AUC). The seed-wise $(\tau_{\text{low}}, \tau_{\text{high}})$ pairs (seeds 1, 7, 21, 42, 100) are $(0.1212, 0.3137)$, $(0.1500, 0.2895)$, $(0.1429, 0.3182)$, $(0.1304, 0.2667)$, $(0.0882, 0.2469)$. All five seed evaluations selected isotonic calibration, consistent with the in-sample selection rule of Section III-C; Brier, NLL, and ECE on the retained test predictions are 0.2155 ± 0.0024, 0.6243 ± 0.0139, and 0.0236 ± 0.0040, respectively.

Within this repetition, threshold-free ROC/PR metrics are much more stable than routing coverage, while specificity is only 0.3014 ± 0.0453. Balanced accuracy, two-class macro-F1, and MCC are 0.6114 ± 0.0107, 0.5532 ± 0.0209, and 0.2782 ± 0.0113. Hence the high vulnerable-class recall should be read as a security-triage operating point with a substantial false-positive burden, not as uniformly improved discrimination.

C. Branch Errors and Cost

The per-record logs permit a direct branch audit of this repetition (Table II). The safety-critical SKIP branch contains few missed vulnerabilities: 23 false negatives among 596 skipped records pooled descriptively across its five seed evaluations. In contrast, the direct-high branch is intentionally aggressive and contains 5,149 false positives among 10,825 high-risk decisions. Because resplit test sets may overlap, pooled counts are descriptive; the mean±SD error rates use the five seed evaluations as the unit of variation.

Within this repetition, more LLM calls did not monotonically improve F1: seed 21 has the largest call ratio (21.89%)

but not the best F1; seeds 42 and 100 use fewer calls with higher F1. Across its five seed evaluations, mean record-level latency is 2.19 ± 0.52 s per test function after feature-store construction, almost all of it on the LLM path. This supports reporting LLM-call ratio as an operational cost proxy, while avoiding the stronger claim that the entire front-end is free.

D. Limitations and Threats to Validity

The reproduced local baseline remains below the published GRACE reference, and no repeated multi-seed local GRACE-style baseline is available; absolute cross-paper differences therefore cannot be attributed to the selective prefilter. The comparison in Table I contrasts seed-averaged results for the selective variant with a single-run baseline and is non-paired (note a of Table I), so its differences and the seed-level consistency check are descriptive and exploratory rather than confirmatory. The run–seed SDs reflect combined split, repetition, training, calibration, and routing variability rather than pure initialization noise. Detailed per-record artifacts are retained only for one five-seed Devign repetition; the remaining measurements retain only metric summaries. Consequently, Big-Vul/ReVeal calibrated-score histograms, matched-recall sweeps, a $\delta_{\text{high}} = 0$ verification run, leave-one-view-out ablations, nested-holdout calibrator selection, and risk–coverage analyses requiring per-example scores need new executions; at approximately 3 hours per Devign or ReVeal run and 24 hours per Big-Vul run (Section IV-C), they are deferred to future work, and the observed dataset dependence is analyzed qualitatively in Section V-A. A complete end-to-end cost audit must also include front-end feature-store construction, which the retained online timing excludes. Finally, the retained raw Devign environment used the heuristic graph fallback because Joern was unavailable, which should be considered when interpreting reproduction gaps and generalization to richer CPG backends.

VI. Conclusion

We presented a calibration-aware selective front-end for a GRACE-style vulnerability-detection pipeline, with published GRACE results as literature context and comparative statements based on a descriptive, non-paired local comparison with a reproduced single-run baseline. In this comparison, mean positive-class F1 and recall, averaged across the three datasets, are higher by 0.0986 and 0.1041 (recall is higher on Devign and Big-Vul but lower on ReVeal), while the fraction of test functions resolved before downstream LLM inference averages 34.59% (equal-weight across the three datasets). Across 25 run–seed measurements per dataset, F1 is 0.6667 ± 0.0053 (Devign), 0.4011 ± 0.0028 (Big-Vul), and 0.5221 ± 0.0052 (ReVeal), with LLM-call ratios of $16.37\% \pm 3.26\%$, $93.18\% \pm 0.90\%$, and $86.68\% \pm 0.74\%$.

The retained five-seed Devign repetition exposes a recall-oriented operating point: recall 0.9214 ± 0.0255 but specificity only 0.3014 ± 0.0453, with a substantial false-positive burden in the direct-high branch. The system should therefore be interpreted as a security-triage mechanism rather than a low-noise scanner, and reductions in downstream LLM usage remain strongly dataset-dependent. Future work should prioritize leave-one-view-out and direct-high ablations, matched-recall and risk–coverage analyses, Big-Vul/ReVeal score-distribution studies, nested-holdout calibrator selection, a repeated local GRACE-style baseline, and complete front-end cost accounting.

References

[1] Y. Zhou, S. Liu, J. Siow, X. Du, and Y. Liu, "Devign: Effective vulnerability identification by learning comprehensive program semantics via graph neural networks," in *Advances in Neural Information Processing Systems*, vol. 32, 2019.

[2] F. Yamaguchi, N. Golde, D. Arp, and K. Rieck, "Modeling and discovering vulnerabilities with code property graphs," in *2014 IEEE Symposium on Security and Privacy*, 2014, pp. 590–604.

[3] S. Chakraborty, R. Krishna, Y. Ding, and B. Ray, "Deep learning based vulnerability detection: Are we there yet?" *IEEE Transactions on Software Engineering*, vol. 48, no. 9, pp. 3280–3296, 2022.

[4] Y. Li, S. Wang, and T. N. Nguyen, "Vulnerability detection with fine-grained interpretations," in *Proc. 29th ACM ESEC/FSE*, 2021, pp. 292–303.

[5] D. Hin, A. Kan, H. Chen, and M. A. Babar, "LineVD: Statement-level vulnerability detection using graph neural networks," in *Proc. 19th Int. Conf. Mining Software Repositories*, 2022, pp. 596–607.

[6] M. Fu and C. Tantithamthavorn, "LineVul: A transformer-based line-level vulnerability prediction," in *Proc. 19th Int. Conf. Mining Software Repositories*, 2022, pp. 608–620.

[7] T. B. Brown *et al.*, "Language models are few-shot learners," in *Advances in Neural Information Processing Systems*, vol. 33, 2020, pp. 1877–1901.

[8] O. Rubin, J. Herzig, and J. Berant, "Learning to retrieve prompts for in-context learning," in *Proc. 2022 NAACL: Human Language Technologies*, 2022, pp. 2655–2671.

[9] G. Lu, X. Ju, X. Chen, W. Pei, and Z. Cai, "GRACE: Empowering LLM-based software vulnerability detection with graph structure and in-context learning," *Journal of Systems and Software*, vol. 212, p. 112031, 2024.

[10] Z. Li *et al.*, "VulDeePecker: A deep learning-based system for vulnerability detection," in *Proc. Network and Distributed System Security Symposium*, 2018.

[11] X. Cheng, H. Wang, J. Hua, G. Xu, and Y. Sui, "DeepWukong: Statically detecting software vulnerabilities using deep graph neural network," *ACM Transactions on Software Engineering and Methodology*, vol. 30, no. 3, pp. 38:1–38:33, 2021.

[12] H. Hanif and S. Maffeis, "VulBERTa: Simplified source code pre-training for vulnerability detection," in *Proc. 2022 Int. Joint Conf. Neural Networks*, 2022, pp. 1–8.

[13] A. Cheshkov, P. Zadorozhny, and R. Levichev, "Evaluation of ChatGPT model for vulnerability detection," arXiv preprint arXiv:2304.07232, 2023.

[14] X. Zhou, T. Zhang, and D. Lo, "Large language model for vulnerability detection: Emerging results and future directions," arXiv preprint arXiv:2401.15468, 2024.

[15] C. Zhang, H. Liu, J. Zeng, K. Yang, Y. Li, and H. Li, "Prompt-enhanced software vulnerability detection using ChatGPT," arXiv preprint arXiv:2308.12697, 2023.

[16] Y. Yang *et al.*, "DLAP: A deep learning augmented large language model prompting framework for software vulnerability detection," *Journal of Systems and Software*, vol. 219, Art. no. 112234, 2025.

[17] Z. Li, S. Dutta, and M. Naik, "LLM-assisted static analysis for detecting security vulnerabilities," arXiv preprint arXiv:2405.17238, 2024.

[18] X. Du *et al.*, "Vul-RAG: Enhancing LLM-based vulnerability detection via knowledge-level RAG," arXiv preprint arXiv:2406.11147, 2024.

[19] Y. Geifman and R. El-Yaniv, "Selective classification for deep neural networks," in *Advances in Neural Information Processing Systems*, vol. 30, 2017.

[20] Y. Geifman and R. El-Yaniv, "SelectiveNet: A deep neural network with an integrated reject option," arXiv preprint arXiv:1901.09192, 2019.

[21] J. C. Platt, "Probabilistic outputs for support vector machines and comparisons to regularized likelihood methods," in *Advances in Large Margin Classifiers*. MIT Press, 1999, pp. 61–74.

[22] A. Niculescu-Mizil and R. Caruana, "Predicting good probabilities with supervised learning," in *Proc. 22nd Int. Conf. Machine Learning*, 2005, pp. 625–632.

[23] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, "On calibration of modern neural networks," in *Proc. 34th Int. Conf. Machine Learning*, 2017, pp. 1321–1330.

[24] J. Fan, Y. Li, S. Wang, and T. N. Nguyen, "A C/C++ code vulnerability dataset with code changes and CVE summaries," in *Proc. 17th Int. Conf. Mining Software Repositories*, 2020, pp. 508–512.

[25] D. Guo, S. Lu, N. Duan, Y. Wang, M. Zhou, and J. Yin, "UniXcoder: Unified cross-modal pre-training for code representation," in *Proc. 60th Annu. Meeting Assoc. Comput. Linguistics*, 2022, pp. 7212–7225.

[26] B. Hui *et al.*, "Qwen2.5-Coder technical report," arXiv preprint arXiv:2409.12186, 2024.